ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA ĐIỆN TỬ - VIỄN THÔNG

BÁO CÁO ĐỒ ÁN

HỌC PHẦN: THỰC HÀNH LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG

TÊN ĐỒ ÁN: XÂY DỰNG HỆ THỐNG QUẢN LÝ NHÀ SÁCH BẰNG OOP

Giảng viên hướng dẫn: Cao Trần Bảo Thương
Nhóm sinh viên thực hiện: Nhóm 21

STT	Họ và tên	MSSV
1	Nguyễn Hoàng Minh Quốc	23207099
2	Phạm Quốc Trung	23207134
3	Lê Bùi Xuân Quang	23207098
4	Phạm Mai Diệu Thảo	23207109
5	Phạm Thái Cẩm Nhung	23207091

Thành phố Hồ Chí Minh - 5/2026

MỤC LỤC

PHẦN MỞ ĐẦU 5
1. Mục tiêu của đồ án 5
2. Phạm vi nghiên cứu và ứng dụng 5
2.1. Phạm vi nghiên cứu 5
2.2. Phạm vi ứng dụng 6
3. Đối tượng sử dụng hệ thống 6
4. Bảng phân công công việc nhóm 7
CHƯƠNG 1: CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ ÁP DỤNG 7
1. Tổng quan về lập trình hướng đối tượng 7
2. Tính đa hình (Polymorphism) và vai trò của nó trong bài toán quản lý 8
2.1. Đa hình thông qua ghi đè phương thức (Method Overriding) 8
2.2. Đa hình thông qua Giao diện (Interface) 9
3. Các công nghệ sử dụng trong hệ thống 9
3.1. Ngôn ngữ lập trình C++ 9
3.2. Giao diện người dùng (HTML, CSS, JavaScript) 9
3.3. Cơ chế kết nối frontend-backend bằng WebView 10
3.4. Lưu trữ dữ liệu (File CSV) 10
CHƯƠNG 2: TỔNG QUAN VỀ HỆ THỐNG QUẢN LÝ NHÀ SÁCH 11
1. Quy trình quản lý nhà sách thực tế 11
2. Các chức năng chính của chương trình 11
2.1. Quản lý sách 11
2.2. Quản lý khách hàng 11
2.3. Quản lý đơn hàng và xử lý mua hàng 12
2.4. Xác thực người dùng và tìm kiếm 12
3. Yêu cầu phi chức năng 12
4. Luồng hoạt động chính của hệ thống 13
CHƯƠNG 3: THIẾT KẾ HỆ THỐNG VÀ TRIỂN KHAI TÍNH ĐA HÌNH 13
1. Kiến trúc tổng thể (Mô hình phân lớp) 13

2. Cấu trúc thư mục của dự án 14
3. Thiết kế lớp (Class Design) 14
3.1. Lớp đối tượng thực thể 14
3.2. Lớp xử lý nghiệp vụ 18
4. Đóng gói vào các lớp đối tượng 20
5. Áp dụng Tính đa hình trong bài toán 21
6. Luồng xử lý dữ liệu 23
CHƯƠNG 4: CÀI ĐẶT VÀ HƯỚNG DẪN SỬ DỤNG 23
1. Môi trường phát triển 23
2. Triển khai các module chính và kết nối giao diện 23
3. Xử lý lưu trữ dữ liệu bằng file CSV 24
4. Hướng dẫn chạy chương trình 24
4.1. Chạy bằng file thực thi 24
4.2. Các lưu ý khi chạy 24
5. Giao diện demo của chương trình 25
KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 29
1. Kết quả đạt được 29
2. Hạn chế của hệ thống 30
3. Hướng phát triển trong tương lai 30
4. Kết luận chung 31
Tài liệu tham khảo 31

DANH MỤC HÌNH ẢNH

Hình 1. Giao diện trang chủ lựa chọn phân hệ người dùng 25
Hình 2. Giao diện đăng nhập cho Quản trị viên.....26
Hình 3. Giao diện quản lý kho sách 26
Hình 4. Giao diện quản lý đơn hàng 27
Hình 5. Giao diện đăng ký thông tin khách hàng.....27
Hình 6. Giao diện tra cứu thông tin sách 28
Hình 7. Giao diện tổng hợp các đơn hàng.....28
Hình 8. Giao diện thông tin chi tiết một đơn hàng.....29

DANH MỤC BẢNG BIỂU

Bảng 1. Bảng phân công công việc 7

PHẦN MỞ ĐẦU

1. Mục tiêu của đồ án

Đồ án "Xây dựng hệ thống quản lý nhà sách bằng OOP" được thực hiện nhằm giải quyết bài toán số hóa quy trình quản lý bán hàng, đồng thời hướng đến các mục tiêu cụ thể sau:

- **Về mặt học thuật và tư duy lập trình:** Củng cố và vận dụng linh hoạt các nguyên lý cốt lõi của Lập trình hướng đối tượng (OOP) trong môi trường C++. Trọng tâm là việc áp dụng tính đóng gói để bảo vệ toàn vẹn dữ liệu, và khai thác tính đa hình nhằm thiết kế các lớp đối tượng (Sách, Khách hàng, Đơn hàng) có khả năng mở rộng cao, dễ dàng đáp ứng các thay đổi về nghiệp vụ mà không làm phá vỡ cấu trúc mã nguồn cũ.

- **Về mặt kỹ thuật và kiến trúc hệ thống:** Xây dựng và làm chủ một kiến trúc phần mềm phân lớp hiện đại. Mục tiêu then chốt là triển khai thành công cơ chế giao tiếp liên mạch giữa backend đảm nhiệm logic nghiệp vụ (viết bằng C++) và frontend hiển thị giao diện người dùng trực quan (HTML, CSS, JavaScript) thông qua cầu nối WebView. Việc thiết kế mô hình này giúp sinh viên nắm bắt sâu sắc cách tách biệt phần xử lý dữ liệu và phần hiển thị (UI).

- **Về mặt ứng dụng thực tiễn:** Hoàn thiện một phần mềm có tính ứng dụng thực tế, cho phép thực hiện trơn tru các quy trình: cập nhật kho sách, tra cứu thông tin khách hàng, xử lý thanh toán đơn hàng và đảm bảo tính nhất quán của dữ liệu qua cơ chế lưu trữ bền vững bằng tập tin định dạng CSV.

2. Phạm vi nghiên cứu và ứng dụng

2.1. Phạm vi nghiên cứu

Đồ án tập trung nghiên cứu và áp dụng các kỹ thuật lập trình phần mềm trong các phạm vi cụ thể sau:

- **Áp dụng nguyên lý Lập trình hướng đối tượng (OOP):** Vận dụng ngôn ngữ C++ để mô hình hóa các thực thể xoay quanh hệ thống nhà sách bao gồm các lớp đối tượng: User, Admin, Customer, Book và Order. Trọng tâm là việc áp dụng tính đóng gói, tính kế thừa và tính đa hình.

- **Kiến trúc phần mềm phân lớp:** Tổ chức mã nguồn theo mô hình phân tách trách nhiệm rõ ràng: Tầng Dữ liệu (Models), Tầng Xử lý nghiệp vụ (Services

như bookService, orderService) và Tầng Điều phối hệ thống (System).

- **Tích hợp giao diện Frontend - Backend:** Nghiên cứu cơ chế nhúng giao diện web (HTML/CSS/JS) vào ứng dụng Desktop thông qua thư viện WebView. Trọng tâm nghiên cứu là xây dựng giao thức truyền thông điệp tùy chỉnh qua hàm callCpp (với cú pháp ACTION|payload) và cơ chế trả về dữ liệu bằng chuỗi JSON.

- **Lưu trữ dữ liệu:** Tìm hiểu và triển khai cơ chế đọc/ghi tệp tin định dạng phẳng (Flat-file) cấu trúc CSV để duy trì tính bền vững của dữ liệu hệ thống.

2.2. Phạm vi ứng dụng

- **Khả năng đáp ứng:** Hệ thống được xây dựng dưới dạng ứng dụng Desktop hoạt động ngoại tuyến (offline), phù hợp để mô phỏng và hỗ trợ quy trình bán hàng cho các nhà sách quy mô nhỏ. Các tính năng bao gồm: quản lý kho sách (thêm, sửa, xóa, tìm kiếm), lưu trữ thông tin khách hàng, xử lý giỏ hàng, tạo đơn hàng và thống kê doanh thu cơ bản.

- **Giới hạn của hệ thống:** Do giới hạn về thời gian và mục tiêu ưu tiên môn học, hệ thống hiện tại sử dụng tệp CSV thay vì các Hệ quản trị cơ sở dữ liệu quan hệ (như MySQL hay SQL Server). Ngoài ra, ứng dụng chưa hỗ trợ đồng bộ dữ liệu đám mây (Cloud) hay tích hợp cổng thanh toán điện tử thực tế.

3. Đối tượng sử dụng hệ thống

Hệ thống quản lý nhà sách phân chia người dùng thành hai nhóm đối tượng chính với các quyền hạn và luồng thao tác độc lập, bao gồm:

- **Quản trị viên (Admin):** Là người điều hành và quản lý trực tiếp các hoạt động dữ liệu của nhà sách. Đối tượng này yêu cầu hệ thống xác thực bảo mật (đăng nhập bằng tài khoản và mật khẩu) trước khi truy cập. Các quyền hạn cốt lõi bao gồm:

  - *Quản lý kho sách:* Thực hiện các thao tác thêm sách mới, cập nhật số lượng tồn kho, và xóa sách khỏi hệ thống.

  - *Quản lý giao dịch:* Theo dõi danh sách toàn bộ đơn hàng, kiểm tra thông tin khách mua và thống kê tổng doanh thu thực tế.

- **Khách hàng (Customer):** Là người dùng cuối (End-user) tiếp cận ứng dụng để

thực hiện nhu cầu mua sắm. Khách hàng không yêu cầu quy trình đăng nhập phức tạp nhưng cần cung cấp thông tin định danh cơ bản (Tên, Giới tính, Số điện thoại) để hệ thống xử lý giao dịch. Các chức năng phục vụ khách hàng bao gồm:

- *Tra cứu sản phẩm:* Tìm kiếm sách theo từ khóa và xem thông tin chi tiết (giá bán, số lượng còn lại).
- *Mua sắm:* Thêm sản phẩm vào giỏ hàng, điều chỉnh số lượng và tiến hành thanh toán (Checkout).
- *Theo dõi cá nhân:* Xem lại lịch sử các đơn hàng đã đặt và trạng thái xử lý của từng đơn hàng.

4. Bảng phân công công việc nhóm

Nhiệm vụ	Thành viên đảm nhiệm
Lên ý tưởng và thiết kế kiến trúc phân lớp cho phần mềm	Nguyễn Hoàng Minh Quốc
Thiết kế giao diện	Phạm Quốc Trung
Thiết kế lớp đối tượng thực	Lê Bùi Xuân Quang
Xây dựng các lớp xử lý dịch vụ	Phạm Mai Diệu Thảo
Xử lý dữ liệu và kết nối Web	Phạm Thái Cẩm Nhung
Kiểm tra toàn bộ luồng chức năng và gỡ lỗi, đóng gói phần mềm	Nguyễn Hoàng Minh Quốc
Viết báo cáo Word	Cả nhóm
Làm báo cáo Powerpoint	Cả nhóm
Thuyết trình	Nguyễn Hoàng Minh Quốc

Bảng 1. Bảng phân công công việc

CHƯƠNG 1: CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ ÁP DỤNG

1. Tổng quan về lập trình hướng đối tượng

Lập trình hướng đối tượng (*Object-Oriented Programming - OOP*) là một mô hình lập trình dựa trên khái niệm về các "đối tượng" (*objects*). Khác với lập trình hướng thủ tục tập trung vào các hàm và luồng xử lý tuần tự, OOP tổ chức thiết kế phần mềm xung quanh dữ liệu (thực thể) thay vì chức năng. Một đối tượng trong OOP được cấu thành từ hai thành phần chính: **Thuộc tính** (trạng thái, dữ liệu) và **Phương thức** (hành vi, chức năng).

Mô hình OOP được xây dựng dựa trên bốn tính chất cốt lõi:

- **Tính trừu tượng (Abstraction):** Là khả năng ẩn đi những chi tiết phức tạp của đối tượng và chỉ hiển thị những thông tin cần thiết nhất đối với người sử dụng. Trong hệ thống nhà sách, thực thể "Sách" được trừu tượng hóa thành lớp Book với các thuộc tính cốt lõi cần quản lý như id, name, type, price, và stock, bỏ qua các thông tin không liên quan đến nghiệp vụ bán hàng (ví dụ: kích thước vật lý của cuốn sách).

- **Tính đóng gói (Encapsulation):** Là cơ chế che giấu dữ liệu nội bộ của một đối tượng nhằm bảo vệ tính toàn vẹn của dữ liệu đó khỏi những can thiệp không hợp lệ từ bên ngoài. Dữ liệu chỉ có thể được truy xuất hoặc thay đổi thông qua các phương thức giao tiếp công khai (*getter* và *setter*).

- **Tính kế thừa (Inheritance):** Cho phép một lớp mới (lớp con) thừa hưởng các thuộc tính và phương thức của một lớp đã tồn tại (lớp cha). Điều này giúp tái sử dụng mã nguồn, giảm thiểu sự dư thừa. Chẳng hạn, trong hệ thống, lớp Admin kế thừa các thông tin cá nhân cơ bản từ lớp cha User (như Tên, Giới tính, Tuổi) và mở rộng thêm các thuộc tính chuyên biệt phục vụ xác thực là account và password.

- **Tính đa hình (Polymorphism):** Cho phép các đối tượng thuộc các lớp khác nhau có thể được xử lý thông qua cùng một giao diện chung. Các lớp con có thể ghi đè (*override*) phương thức của lớp cha để cung cấp những hành vi mang tính đặc thù.

2. Tính đa hình (Polymorphism) và vai trò của nó trong bài toán quản lý

Tính đa hình (*Polymorphism*) là một trong những đặc trưng quan trọng nhất của lập trình hướng đối tượng. Về mặt lý thuyết, nó cho phép các đối tượng thuộc các lớp khác nhau có thể thực thi cùng một phương thức nhưng theo những cách thức mang tính đặc thù của riêng từng đối tượng. Cơ chế này giúp hệ thống trở nên linh hoạt, tối ưu hóa khả năng tái sử dụng mã nguồn và giảm thiểu sự phụ thuộc phức tạp (coupling) giữa các thành phần.

Trong bài toán quản lý nhà sách, tính đa hình đóng vai trò là "xương sống" cho kiến trúc phần mềm, được ứng dụng ở hai khía cạnh chính:

2.1. Đa hình thông qua ghi đè phương thức (Method Overriding)

Hệ thống phân chia người dùng thành các vai trò quản trị (Admin) và khách

hàng (Customer) với những đặc quyền khác nhau, nhưng họ vẫn chia sẻ một số đặc tính chung của một người dùng (User). Bằng cách áp dụng đa hình, hệ thống cho phép định nghĩa một phương thức chung để lấy vai trò. Tuy nhiên, khi được gọi, tùy thuộc vào đối tượng thực tế đang tồn tại trong bộ nhớ là Admin hay User thông thường, hệ thống sẽ tự động liên kết động (*dynamic binding*) để trả về định danh phân quyền chính xác. Điều này giúp tầng Giao diện (Frontend) dễ dàng nhận diện và điều hướng đúng luồng thao tác.

2.2. Đa hình thông qua Giao diện (Interface)

Trong quá trình vận hành, hệ thống cần xử lý nhiều loại dữ liệu khác nhau như: lưu trữ danh sách sách, lưu trữ khách hàng, và quản lý các hóa đơn giao dịch. Thay vì phải thiết kế các hàm đọc/ghi tệp tin rời rạc và lặp lại cho từng loại dữ liệu, tính đa hình được áp dụng thông qua một Giao diện chung (*Interface*).

Giao diện này đóng vai trò như một "hợp đồng" quy định sẵn các thao tác cần thiết. Tầng Điều phối trung tâm chỉ cần giao tiếp với hợp đồng này mà không cần quan tâm chi tiết bên trong đang xử lý tệp tin sách hay tệp tin đơn hàng. Vai trò của đa hình ở đây là giúp mã nguồn trở nên gọn nhẹ; đồng thời, khi nhà sách cần mở rộng thêm các dịch vụ mới trong tương lai, lập trình viên có thể tích hợp chúng vào hệ thống một cách trơn tru mà không làm phá vỡ kiến trúc cốt lõi hiện tại.

3. Các công nghệ sử dụng trong hệ thống

3.1. Ngôn ngữ lập trình C++

C++ là ngôn ngữ cốt lõi được sử dụng để xây dựng toàn bộ logic nghiệp vụ (Backend) của hệ thống. Đây là ngôn ngữ mạnh mẽ, cho phép can thiệp sâu vào bộ nhớ và đặc biệt hỗ trợ hoàn hảo cho mô hình Lập trình hướng đối tượng (OOP). Trong đồ án này, C++ đảm nhiệm các vai trò:

- Mô hình hóa dữ liệu thông qua các lớp đối tượng (Class).
- Xử lý logic tính toán (ví dụ: tính tổng tiền giỏ hàng, cập nhật số lượng tồn kho).
- Quản lý luồng đọc/ghi dữ liệu từ bộ nhớ ra tập tin thông qua các thư viện chuẩn như <fstream> và <sstream>.

3.2. Giao diện người dùng (HTML, CSS, JavaScript)

Để khắc phục hạn chế về mặt đồ họa của C++ truyền thống, hệ thống sử dụng

bộ ba công nghệ Web (HTML5, CSS3, JavaScript) để xây dựng giao diện người dùng (Frontend).

- **HTML & CSS:** Thiết kế bố cục rõ ràng, chia làm 3 phân hệ độc lập (Trang chủ, Khách hàng, Quản trị viên). CSS Flexbox được sử dụng để căn chỉnh thành phần giao diện một cách linh hoạt mà không cần phụ thuộc vào các framework nặng nề.
- **JavaScript (Vanilla JS):** Đóng vai trò xử lý sự kiện (nhấp chuột, tìm kiếm, tính toán giỏ hàng) và thao tác trực tiếp với giao diện (DOM Manipulation) giúp cập nhật dữ liệu hiển thị tức thời mà không cần tải lại toàn bộ trang.

3.3. Cơ chế kết nối frontend-backend bằng WebView

Điểm nhấn kỹ thuật của đồ án là việc tích hợp C++ và giao diện Web thành một ứng dụng Desktop hoàn chỉnh thông qua công nghệ WebView. Cơ chế này hoạt động dựa trên giao tiếp hai chiều:

- **Frontend gọi Backend:** JavaScript thu thập dữ liệu từ người dùng và gửi thông điệp dạng chuỗi (ví dụ: ACTION|payload) qua phương thức window.callCpp() xuống tầng C++ để xử lý.
- **Backend cập nhật Frontend:** Sau khi C++ xử lý xong dữ liệu (từ file hoặc bộ nhớ), nó đóng gói kết quả thành định dạng JSON và dùng phương thức w.eval() để tiêm (inject) dữ liệu ngược trở lại các hàm JavaScript hiển thị trên giao diện.

3.4. Lưu trữ dữ liệu (File CSV)

Thay vì sử dụng các Hệ quản trị cơ sở dữ liệu lớn (như MySQL hay SQL Server) đòi hỏi cài đặt phức tạp, hệ thống lựa chọn sử dụng tập tin định dạng CSV (Comma Separated Values) để lưu trữ dữ liệu.

- **Đặc điểm:** Dữ liệu sách, khách hàng và đơn hàng được lưu dưới dạng văn bản thuần túy (plain text), các trường dữ liệu cách nhau bởi dấu phẩy.
- **Ưu điểm:** Phương pháp này cực kỳ nhẹ, tốc độ truy xuất nhanh đối với dữ liệu quy mô nhỏ và vừa. Nó cho phép sinh viên vận dụng tốt các kỹ năng xử lý chuỗi (getline, stringstream phân tách chuỗi) và luồng tệp tin (ifstream, ofstream) thuần túy của C++ để thao tác với dữ liệu.

CHƯƠNG 2: TỔNG QUAN VỀ HỆ THỐNG QUẢN LÝ NHÀ SÁCH

1. Quy trình quản lý nhà sách thực tế

Trong thực tế, quy trình vận hành của một nhà sách đòi hỏi sự phối hợp nhịp nhàng giữa khâu quản lý kho bãi và khâu bán hàng. Khi nhập sách mới, người quản lý cần ghi nhận các thông tin định danh như mã sách, tên sách, thể loại, giá bán và số lượng tồn kho. Khi khách hàng đến mua sắm, họ sẽ tra cứu sách theo nhu cầu, chọn mua và tiến hành thanh toán. Tại quầy thu ngân, nhân viên ghi nhận thông tin khách hàng, lập hóa đơn, tính tổng tiền và cập nhật lại số lượng sách còn lại trong kho. Cuối cùng, quản lý nhà sách sẽ xem xét các báo cáo đơn hàng để thống kê doanh thu. Hệ thống phần mềm được phát triển trong đồ án này nhằm mục đích số hóa và tự động hóa toàn bộ chuỗi quy trình khép kín nêu trên.

2. Các chức năng chính của chương trình

2.1. Quản lý sách

Đây là phân hệ cốt lõi dành cho Quản trị viên (Admin), cho phép số hóa toàn bộ dữ liệu kho sách. Các chức năng bao gồm:

- **Thêm sách mới:** Ghi nhận thông tin chi tiết của một đầu sách (Tên, Thể loại, Giá, Số lượng) và tự động cấp phát ID.
- **Cập nhật tồn kho:** Cho phép tăng hoặc giảm số lượng sách trong kho dựa trên ID sách khi có biến động nhập/xuất thực tế.
- **Xóa sách:** Loại bỏ hoàn toàn một đầu sách khỏi hệ thống lưu trữ khi không còn kinh doanh.

2.2. Quản lý khách hàng

Hệ thống yêu cầu khách hàng cung cấp thông tin định danh trước khi tiến hành mua sắm.

- **Lưu trữ thông tin:** Ghi nhận các trường dữ liệu bao gồm Tên, Giới tính và Số điện thoại.
- **Định danh tự động:** Hệ thống tự động khởi tạo mã khách hàng (Customer ID) bằng kỹ thuật timestamp (dựa trên thời gian thực) để đảm bảo tính duy nhất của mỗi khách hàng trong hệ thống.

2.3. Quản lý đơn hàng và xử lý mua hàng

Phân hệ này phục vụ luồng giao dịch tài chính của hệ thống, bao gồm:

- **Quản lý giỏ hàng:** Khách hàng có thể thêm nhiều đầu sách khác nhau vào giỏ hàng và tùy chỉnh số lượng mua.
- **Xử lý thanh toán (Checkout):** Khi khách hàng chốt đơn, hệ thống tự động tính toán tổng số tiền (dựa trên đơn giá và số lượng từng loại sách). Đồng thời, hệ thống tự động trừ đi số lượng tồn kho tương ứng của các sách đã mua trong cơ sở dữ liệu kho.
- **Lịch sử và Thống kê:** Khách hàng có thể xem lại lịch sử các đơn hàng cá nhân. Về phía Admin, hệ thống hiển thị toàn bộ danh sách giao dịch, trạng thái đơn hàng (Pending/Done) và tự động tính toán tổng doanh thu của nhà sách.

2.4. Xác thực người dùng và tìm kiếm

- **Xác thực (Authentication):** Để đảm bảo an toàn dữ liệu, phân hệ Quản trị viên được bảo vệ bằng cơ chế đăng nhập. Quản trị viên phải nhập chính xác tài khoản và mật khẩu đã được định nghĩa trong đối tượng Admin thì mới được phép truy cập vào bảng điều khiển (Dashboard).
- **Tìm kiếm (Search):** Cung cấp công cụ tìm kiếm sách theo tên cho cả Admin và Customer. Thuật toán tìm kiếm được tối ưu bằng cách chuẩn hóa toàn bộ chuỗi ký tự về dạng chữ thường (lowercase) trước khi so sánh, giúp mang lại kết quả chính xác không phân biệt chữ hoa, chữ thường.

3. Yêu cầu phi chức năng

Bên cạnh các chức năng nghiệp vụ, hệ thống còn đáp ứng các tiêu chuẩn kỹ thuật sau:

- **Trải nghiệm người dùng (UX/UI):** Giao diện phải trực quan, dễ thao tác. Nhờ kết hợp JavaScript trên nền WebView, các thao tác tính toán giỏ hàng, tra cứu hay cập nhật giao diện được thực hiện tức thời trên DOM mà không gây độ trễ do phải tải lại trang (Reload).
- **Tính toàn vẹn dữ liệu:** Dữ liệu phải được lưu trữ bền vững (persistent). Hệ thống sử dụng cơ chế đọc/ghi tệp tin phẳng (CSV) liên tục để đảm bảo thông tin sách, khách hàng và đơn hàng không bị mất đi khi đóng ứng dụng.

4. Luồng hoạt động chính của hệ thống

Hệ thống cung cấp một màn hình chọn vai trò (Trang chủ) ngay khi khởi động. Từ đây, luồng hoạt động chia làm hai nhánh độc lập:

- **Luồng hoạt động của Quản trị viên (Admin Workflow):**
  - Từ Trang chủ, chọn quyền Admin.
  - Nhập tài khoản và mật khẩu tại màn hình Đăng nhập.
  - Nếu xác thực thành công, hệ thống chuyển hướng đến Dashboard.
  - Tại đây, Admin có thể thực hiện thao tác quản lý kho (Thêm, Sửa tồn kho, Xóa, Tìm kiếm sách) hoặc xem danh sách Đơn hàng và thống kê doanh thu.
  - Kết thúc phiên làm việc bằng thao tác Đăng xuất.
- **Luồng hoạt động của Khách hàng (Customer Workflow):**
  - Từ Trang chủ, chọn quyền Customer.
  - Khai báo thông tin cá nhân (Tên, SĐT, Giới tính) để định danh.
  - Hệ thống chuyển hướng vào giao diện Cửa hàng (Shop). Khách hàng tìm kiếm sách và thêm sản phẩm vào Giỏ hàng.
  - Mở Giỏ hàng, kiểm tra tổng tiền và nhấn "Thanh toán".
  - Hệ thống xử lý đơn hàng, trừ tồn kho và ghi nhận vào tệp CSV. Khách hàng có thể theo dõi lại đơn trong mục "Đơn hàng của tôi".

CHƯƠNG 3: THIẾT KẾ HỆ THỐNG VÀ TRIỂN KHAI TÍNH ĐA HÌNH

1. Kiến trúc tổng thể (Mô hình phân lớp)

Hệ thống quản lý nhà sách được thiết kế theo kiến trúc phân lớp (Layered Architecture) nhằm đảm bảo nguyên lý Đơn trách nhiệm (Single Responsibility Principle), giúp mã nguồn dễ bảo trì và mở rộng. Kiến trúc bao gồm 3 tầng chính:

- **Tầng Giao diện (Frontend/UI):** Xây dựng bằng HTML/CSS/JS, chịu trách nhiệm hiển thị dữ liệu và tiếp nhận tương tác của người dùng (Customer, Admin).
- **Tầng Điều phối (Controller):** Lớp SystemApp đóng vai trò trung tâm định tuyến

(router), sử dụng WebView để lắng nghe các sự kiện từ UI thông qua callCpp, sau đó điều phối lệnh xuống tầng dưới.

- **Tầng Xử lý nghiệp vụ & Dữ liệu (Services & Models):** Nơi chứa toàn bộ logic phần mềm bằng C++. Tầng này thực hiện các phép toán, truy xuất dữ liệu đối tượng và giao tiếp với tập tin CSV.

2. Cấu trúc thư mục của dự án

Dự án được tổ chức gọn gàng, tách biệt mã nguồn logic, giao diện và dữ liệu theo cấu trúc cây thư mục như sau:

- data/: Thư mục chứa cơ sở dữ liệu định dạng phẳng bao gồm customer.csv, data.csv (lưu sách), và orders.csv.
- UI/: Chứa các tệp giao diện người dùng (admin-dashboard.html, admin-login.html, customer.html, UI.html) và các tệp hình ảnh tĩnh (admin.png, book.png, customer.png).
- src/: Thư mục gốc chứa toàn bộ mã nguồn C++.
  - src/models/: Định nghĩa các lớp thực thể lưu trữ cấu trúc dữ liệu (Admin.cpp, book.cpp, Customer.cpp, order.cpp, User.cpp) và thư mục include/ chứa các tệp tiêu đề (header .hpp) tương ứng.
  - src/services/: Chứa các lớp xử lý nghiệp vụ (bookService.cpp, customerService.cpp, orderService.cpp, System.cpp) và thư mục include/ chứa tệp tiêu đề định nghĩa Interface cùng các dịch vụ.
- **Thành phần thực thi (Root):** Bao gồm tệp biên dịch tự động Makefile, tệp chạy ứng dụng NhaSachFetel.exe, tệp thư viện liên kết động WebView2Loader.dll và tài liệu hướng dẫn Readme.md.

3. Thiết kế lớp (Class Design)

3.1. Lớp đối tượng thực thể

Các lớp thực thể được thiết kế để đại diện cho các đối tượng trong thế giới thực:

- **Lớp Book:** Lưu trữ thông tin sách với các trường id, Name, Type, price, stock.

```
#pragma once
```

```
#include <string>
using namespace std;
class Book
{
private:
    string id;
    string Name;
    string Type;
    float price;
    int stock;
public:
    // Constructor
    Book(const string &id, const string &Name,
        const string &Type, float price, int stock);
    // Setter
    void setName(const string &Name);
    void setType(const string &Type);
    void setPrice(float price);
    void setStock(int stock);
    void setId(const string &id);
    // Getter
    string getName() const;
    string getType() const;
    float getPrice() const;
    int getStock() const;
    string getId() const;
};
```

- **Lớp User và Admin:** Quản lý thông tin định danh. Lớp Admin kế thừa từ User để tái sử dụng các thuộc tính cá nhân (tên, giới tính, tuổi) và bổ sung thêm thông tin đăng nhập (tài khoản, mật khẩu).

```
#pragma once
#include <string>
#include <iostream>
using namespace std;
class User
{
protected:
    string Name;
    string gender;
    int age;
public:
    // Constructor
    User(const string &Name, const string &gender, int age);
```

```
    // Setter
    void setName(const string &Name);
    void setGender(const string &gender);
    void setAge(int age);
    // Getter
    string getName() const;
    string getGender() const;
    int getAge() const;
    // Behavior
    void displayInfo() const;
    // Polymorphism
    virtual string getRole() const;
};
#pragma once
#include <string>
using namespace std;
#include "User.hpp"
class Admin : public User
{
private:
    string account;
    string password;
public:
    // Constructor
    Admin(const string &Name, const string &gender, int age,
        const string &account = "admin",
        const string &password = "123");
    // Getter
    string getAccount() const;
    string getPassword() const;
    // Setter
    void setAccount(const string &acc);
    void setPassword(const string &pass);
    // Login
    bool login(const string &username, const string &pass) const;
    // Polymorphism
    string getRole() const override;
};
```

- **Lớp Customer:** Lưu trữ thông tin khách mua hàng độc lập (name, gender, phone).

```
#pragma once
#include <string>
using namespace std;
```

```
class Customer
{
private:
    string id;
    string name;
    string gender;
    string phone;
public:
    // Constructor
    Customer();
    Customer(const string &id, const string &name,
            const string &gender, const string &phone);
    // Getter
    string getId() const;
    string getName() const;
    string getGender() const;
    string getPhone() const;
    // Setter (thêm để chuẩn OOP)
    void setId(const string &id);
    void setName(const string &name);
    void setGender(const string &gender);
    void setPhone(const string &phone);
};
```

- **Lớp Order và OrderItem:** Lớp Order quản lý thông tin hóa đơn (mã đơn, mã khách, trạng thái, tổng tiền) và chứa một danh sách (vector) các OrderItem đại diện cho các chi tiết cấu thành đơn hàng.

```
#pragma once
#include <string>
#include <vector>
using namespace std;
struct OrderItem
{
    string bookId;
    string name;
    int quantity;
    double price;
};
class Order
{
private:
    string orderId;
    string customerId;
    vector<OrderItem> items;
```

```
    double totalAmount;
    string status;
public:
    // Constructor
    Order();
    Order(const string &oId, const string &cId,
          const vector<OrderItem> &items,
          const string &status);
    // Getter
    string getOrderId() const;
    string getCustomerId() const;
    double getTotalAmount() const;
    string getStatus() const;
    const vector<OrderItem> &getItems() const;
    // Setter
    void setStatus(const string &newStatus);
    void setItems(const vector<OrderItem> &newItems);
    // Behavior (OOP chuẩn)
    void addItem(const OrderItem &item);
    // Logic
    void calculateTotal();
};
```

3.2. Lớp xử lý nghiệp vụ

Tầng này cung cấp các phương thức thao tác trực tiếp với dữ liệu:

- **Lớp bookService:** Xử lý logic tìm kiếm, thêm, xóa sách và cập nhật tồn kho. Phụ trách đọc/ghi tệp data.csv.

```
#pragma once
#include "book.hpp"
#include "IService.hpp"
#include <vector>
#include <fstream>
#include <sstream>
using namespace std;
class bookService : public IService
{
private:
    vector<Book> books;
public:
    // CRUD
    void addBook(const string &id, const string &Name,
                 const string &Type, float price, int stock);
```

```
    bool updateStock(const string &id, int additionalStock);
    bool deleteBook(const string &id);
    // Getter
    vector<Book> &getAllBooks();
    vector<Book> searchBookByTitle(const string &keyword);
    Book *getBookById(const string &bookId);
    // File (đa hình từ IService)
    void loadFromFile(string filepath) override;
    void saveToFile(string filepath) override;
};
```

- **Lớp customerService:** Quản lý danh sách khách hàng và đọc/ghi tệp customer.csv.

```
#pragma once
#include "Customer.hpp"
#include "IService.hpp"
#include <vector>
#include <string>
using namespace std;
class customerService : public IService
{
private:
    vector<Customer> customers;
public:
    // CRUD
    void addCustomer(const string &id,
                     const string &name,
                     const string &gender,
                     const string &phone);
    vector<Customer> getAllCustomers() const;
    // File (đa hình từ IService)
    void loadFromFile(string filepath) override;
    void saveToFile(string filepath) override;
};
```

- **Lớp OrderService:** Tính toán doanh thu, cập nhật trạng thái đơn và giao tiếp với orders.csv.

```
#pragma once
#include "order.hpp"
#include "IService.hpp"
#include <vector>
#include <fstream>
#include <sstream>
#include <string>
```

```
using namespace std;
class OrderService : public IService
{
private:
    vector<Order> orders;
public:
    // File (đa hình từ IService)
    void loadFromFile(string filepath) override;
    void saveToFile(string filepath) override;
    // Getter
    vector<Order> getAllOrders() const;
    // CRUD
    void addOrder(const Order &order);
    void updateStatus(const string &orderId, const string &newStatus);
};
```

- **Interface IService:** Lớp trừu tượng (Abstract class) quy định các phương thức chuẩn như loadFromFile và saveToFile bắt buộc các lớp Service khác phải tuân thủ.

```
#pragma once
#include <string>
using namespace std;
class IService
{
public:
    // đa hình (pure virtual)
    virtual void loadFromFile(string filepath) = 0;
    virtual void saveToFile(string filepath) = 0;
    // destructor ảo (chuẩn OOP)
    virtual ~IService() {}
};
```

4. Đóng gói vào các lớp đối tượng

Tính đóng gói (Encapsulation) được tuân thủ nghiêm ngặt trong toàn bộ mã nguồn. Tại các lớp thực thể (Book, Customer, Order, Admin), tất cả các thuộc tính dữ liệu đều được khai báo với phạm vi truy cập private để ngăn chặn việc sửa đổi dữ liệu tùy tiện từ bên ngoài. Để giao tiếp với các dữ liệu này, hệ thống cung cấp các phương thức public (Getter và Setter).

Minh chứng rõ nét nhất là tại lớp Admin, dữ liệu nhạy cảm là tài khoản và mật khẩu được bảo vệ khép kín. Các tầng khác không thể trích xuất trực tiếp biến password mà bắt buộc phải thông qua phương thức login(username, pass) để xác thực.

Đoạn mã chứng minh tính đóng gói trong lớp Admin (Trích file Admin.cpp):

```
// ================= LOGIN =================
bool Admin::login(const string &username, const string &pass) const
{
    return username == account && pass == password;
}
```

5. Áp dụng Tính đa hình trong bài toán

Hệ thống triển khai tính đa hình thông qua hai phương pháp chính nhằm tối ưu hóa khả năng mở rộng:

Đa hình thời gian chạy (Runtime Polymorphism - Overriding):

- Hệ thống phân chia người dùng thành các vai trò khác nhau nhưng có chung một số đặc điểm cơ bản. Lớp Admin được kế thừa từ lớp cha User. Tính đa hình thể hiện rõ nhất qua việc ghi đè phương thức getRole().
- Dù được gọi thông qua cùng một cấu trúc hàm, mỗi đối tượng sẽ trả về một định danh vai trò riêng biệt, hỗ trợ hệ thống phân quyền giao diện ở tầng WebView.

Đoạn mã minh họa trong User.cpp và Admin.cpp:

```
// Trong lớp cha User (User.cpp)
string User::getRole() const {
    return "User";
}
// Trong lớp con Admin kế thừa từ User (Admin.cpp)
string Admin::getRole() const {
    return "Admin";
```

Đa hình qua Giao diện (Interface):

- Đây là vai trò lớn nhất của tính đa hình trong bài toán quản lý nhà sách. Hệ thống có 3 dịch vụ khác nhau: bookService, customerService, và orderService. Thay vì phải viết 3 hàm tải/lưu dữ liệu riêng biệt cho từng dịch vụ, hệ thống sử dụng một Interface mang tên IService chứa các phương thức ảo (virtual methods) để xử lý file.
- Trong file System.cpp, một cấu trúc từ điển (map) được sử dụng để lưu trữ các con trỏ kiểu IService*. Nhờ cơ chế liên kết động (dynamic binding) của tính đa hình, chỉ cần một hàm load duy nhất, hệ thống có thể tự động hiểu và gọi đúng phương thức đọc file của từng dịch vụ tương ứng dựa trên key truyền vào.

Đoạn mã minh họa trong System.cpp và IService.hpp thể hiện vai trò của đa

hình:

```
class IService
{
public:
    // đa hình (pure virtual)
    virtual void loadFromFile(string filepath) = 0;
    virtual void saveToFile(string filepath) = 0;
    // destructor ảo (chuẩn OOP)
    virtual ~IService() {}
};
```

```
// ================= HELPER POLYMORPHISM =================
void load(map<string, IService *> &m, const string &key, const string &file)
{
    m[key]->loadFromFile(file);
}
void save(map<string, IService *> &m, const string &key, const string &file)
{
    m[key]->saveToFile(file);
}
    // ===== POLYMORPHISM =====
    serviceMap["book"] = &bs;
    serviceMap["customer"] = &cs;
    serviceMap["order"] = &os;
        if (action == "GET_BOOKS_CUSTOMER")
        {
            load(serviceMap, "book", "data/data.csv");
            w.eval("renderBookPage(" + booksToJSON(bs.getAllBooks()) + ");");
        }
        if (action == "GET_ORDERS")
        {
            load(serviceMap, "order", "data/orders.csv");
            double revenue = 0;
            for (auto &o : os.getAllOrders())
                if (o.getStatus() == "Done")
                    revenue += o.getTotalAmount();
            w.eval("renderOrderPage(" +
                ordersToJSON(os.getAllOrders()) + "," +
                to_string(revenue) + ");");
        }
```

- Thông qua việc áp dụng tính đa hình bằng IService, khi bài toán thực tế yêu cầu mở rộng thêm các dịch vụ mới (ví dụ: employeeService, discountService), lập

trình viên chỉ cần tạo lớp mới kế thừa IService và đăng ký vào serviceMap mà không cần sửa đổi hay viết thêm các hàm đọc/ghi dữ liệu ở tầng hệ thống trung tâm.

6. **Luồng xử lý dữ liệu**

Dữ liệu di chuyển theo luồng khép kín (Data Flow) qua các tầng kiến trúc như sau:

- Người dùng thao tác trên giao diện Web (HTML/JS), dữ liệu đầu vào được thu thập và đóng gói thành chuỗi văn bản phân tách bằng ký tự | (Ví dụ: ADD|Sách A|IT|50000|10).
- Hàm JavaScript gọi window.callCpp() truyền chuỗi này xuống ứng dụng Desktop.
- Lớp SystemApp (C++) phân rã chuỗi, nhận diện hành động (Ví dụ: ADD) và gọi phương thức addBook() của đối tượng bookService.
- bookService khởi tạo đối tượng Book mới, đưa vào danh sách quản lý và tiến hành gọi hàm saveToFile() để ghi ngay dữ liệu vào file data.csv.
- SystemApp gửi lệnh w.eval("location.reload();") yêu cầu giao diện làm mới, hoàn tất một chu kỳ xử lý an toàn và đồng bộ.

CHƯƠNG 4: CÀI ĐẶT VÀ HƯỚNG DẪN SỬ DỤNG

1. **Môi trường phát triển**

Hệ thống được phát triển và kiểm thử trên môi trường hệ điều hành Windows. **Các công cụ và bộ thư viện cốt lõi bao gồm:**

- **Trình biên dịch (Compiler):** Sử dụng trình biên dịch GCC/G++ (thông qua môi trường MinGW) để biên dịch mã nguồn C++ thành tập tin thực thi.
- **Thư viện đồ họa:** Tích hợp bộ thư viện Microsoft Edge WebView2 SDK để kết xuất (render) giao diện Web ngay bên trong cửa sổ ứng dụng Desktop C++.
- **Công cụ phát triển giao diện:** Sử dụng các trình duyệt web hiện đại và công cụ kiểm thử (Developer Tools) để xây dựng HTML/CSS/JS.

2. **Triển khai các module chính và kết nối giao diện**

Quá trình triển khai dự án được tự động hóa để đảm bảo tính đồng bộ. Thay vì phải biên dịch thủ công từng tập tin .cpp riêng lẻ, dự án sử dụng tệp lệnh Makefile để tự động hóa quá trình liên kết (linking) các module nghiệp vụ (bookService,

orderService...) với thư viện WebView2Loader. Khi ứng dụng khởi chạy, lớp SystemApp ở tầng C++ sẽ khởi tạo một cửa sổ Windows bản địa, sau đó kích hoạt engine WebView2 và nạp tệp giao diện gốc theo đường dẫn tương đối: file:///.../UI/UI.html. Từ thời điểm này, kết nối hai chiều giữa mã nguồn C++ và giao diện HTML chính thức được thiết lập.

3. Xử lý lưu trữ dữ liệu bằng file CSV

Cơ chế lưu trữ được thiết kế theo hướng đảm bảo an toàn dữ liệu cao nhất cho người dùng:

- **Tải dữ liệu (Load):** Khi ứng dụng vừa khởi động hoặc khi người dùng chuyển hướng trang, hệ thống sẽ đọc dữ liệu từ các tập tin trong thư mục data/ (data.csv, customer.csv, orders.csv), phân tách chuỗi và nạp vào bộ nhớ RAM dưới dạng các danh sách đối tượng (Vector) để sẵn sàng truy xuất.
- **Lưu dữ liệu (Save):** Bất cứ khi nào có một thao tác thay đổi trạng thái dữ liệu (Thêm sách, Cập nhật tồn kho, Tạo đơn hàng thành công), hệ thống sẽ lập tức gọi hàm saveToFile() để ghi đè toàn bộ danh sách đối tượng hiện tại trở lại tập tin CSV tương ứng. Cơ chế ghi tức thời này giúp hệ thống không bị mất dữ liệu ngay cả khi ứng dụng bị đóng đột ngột.

4. Hướng dẫn chạy chương trình

4.1. Chạy bằng file thực thi

Để trải nghiệm phần mềm mà không cần biên dịch lại mã nguồn, người dùng thực hiện theo các bước sau:

- Truy cập vào thư mục gốc của dự án đã được giải nén.
- Tìm và nhấp đúp chuột vào tập tin thực thi chính: NhaSachFetel.exe.
- Ứng dụng sẽ tự động mở lên màn hình Trang chủ để người dùng lựa chọn vai trò (Admin hoặc Customer).
- *(Lưu ý: Đối với lập trình viên muốn biên dịch lại mã nguồn, có thể sử dụng môi trường dòng lệnh terminal và gõ lệnh make).*

4.2. Các lưu ý khi chạy

Để hệ thống hoạt động ổn định và không phát sinh lỗi, môi trường chạy cần đáp ứng các điều kiện tiên quyết sau:

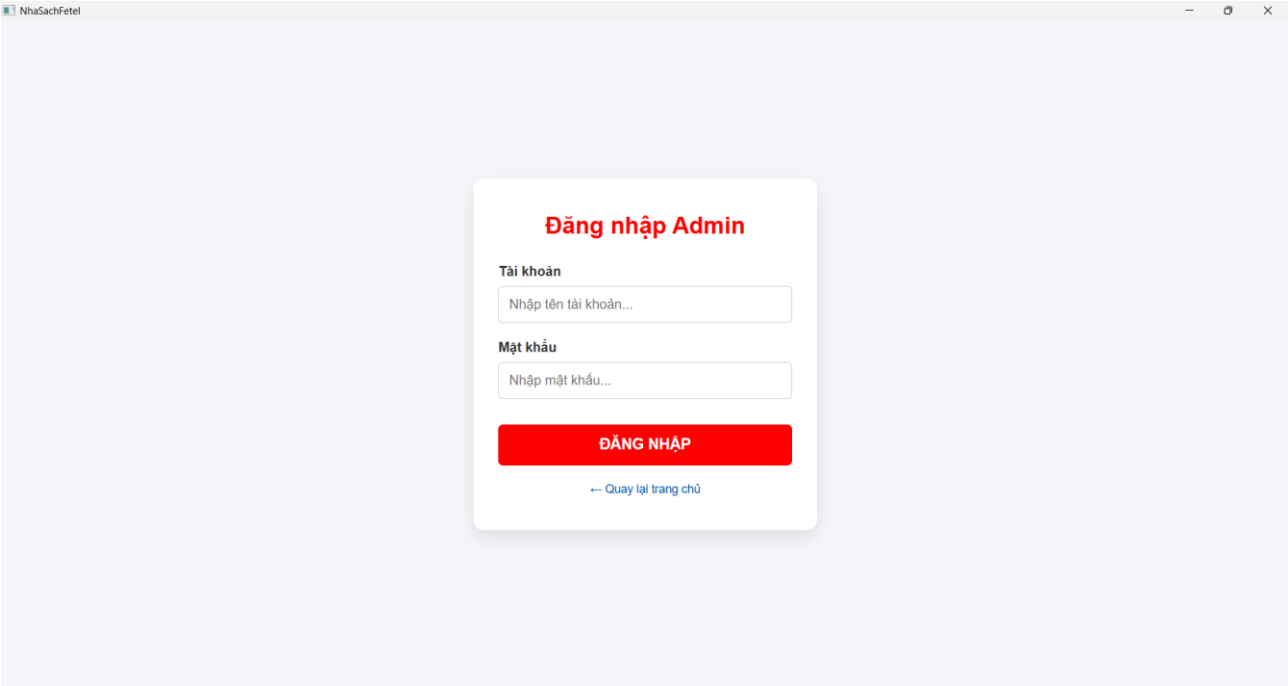
- **Thư viện liên kết động:** Tập tin WebView2Loader.dll bắt buộc phải nằm cùng cấp thư mục với tập tin NhaSachFetel.exe.
- **Cấu trúc thư mục:** Không được thay đổi tên hay di chuyển các thư mục UI/ và data/. Ứng dụng sử dụng đường dẫn tương đối, việc thay đổi cấu trúc sẽ khiến hệ thống không tìm thấy giao diện hoặc không thể nạp cơ sở dữ liệu.
- **Môi trường hệ điều hành:** Máy tính Windows cần được cài đặt sẵn WebView2 Runtime (thường được cài đặt mặc định trên các phiên bản Windows 10 và 11 hiện tại).

5. Giao diện demo của chương trình

Giao diện Trang chủ lựa chọn phân hệ người dùng.

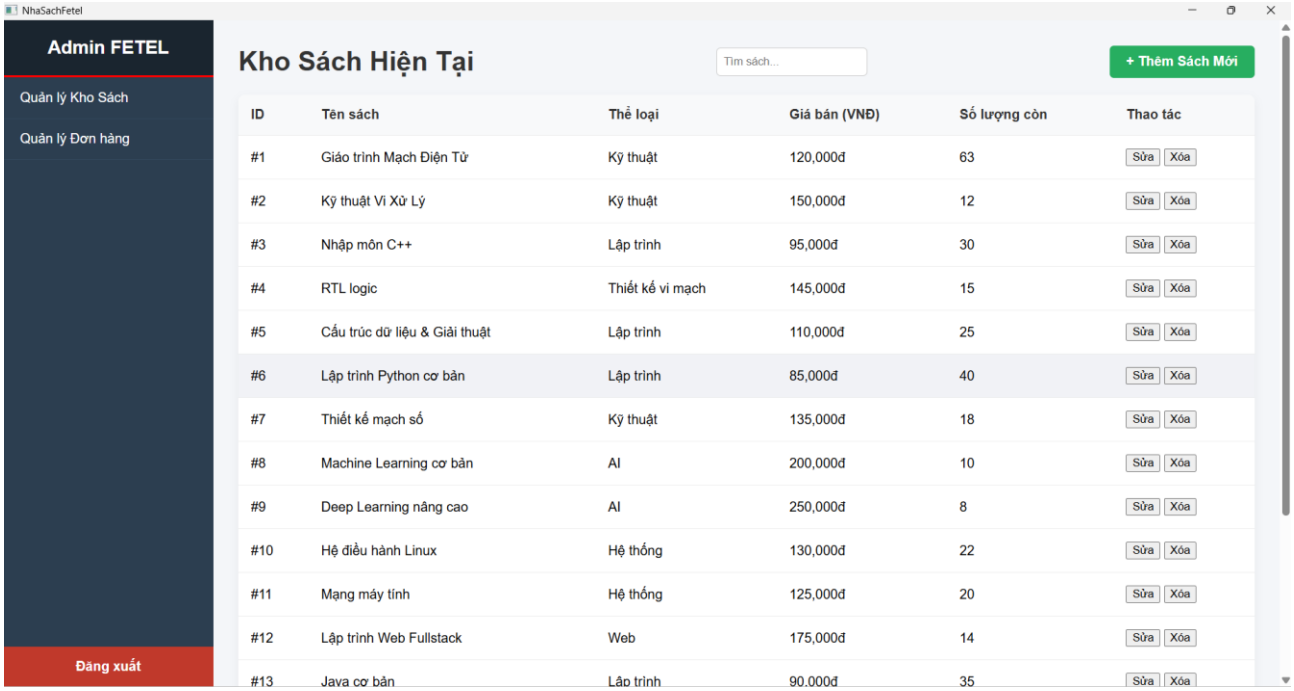
Hình 1. Giao diện trang chủ lựa chọn phân hệ người dùng

Giao diện đăng nhập dành cho Quản trị viên (Admin).

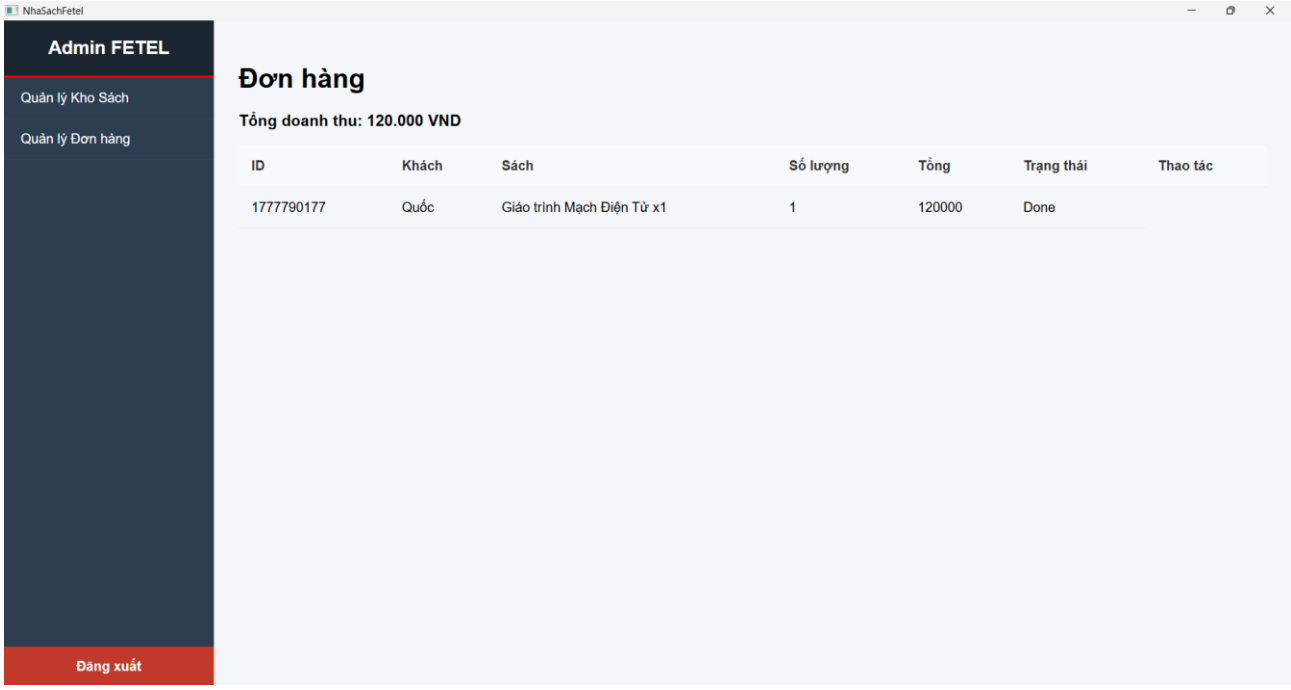


Hình 2. Giao diện đăng nhập cho Quản trị viên

Giao diện bảng điều khiển quản lý kho sách.

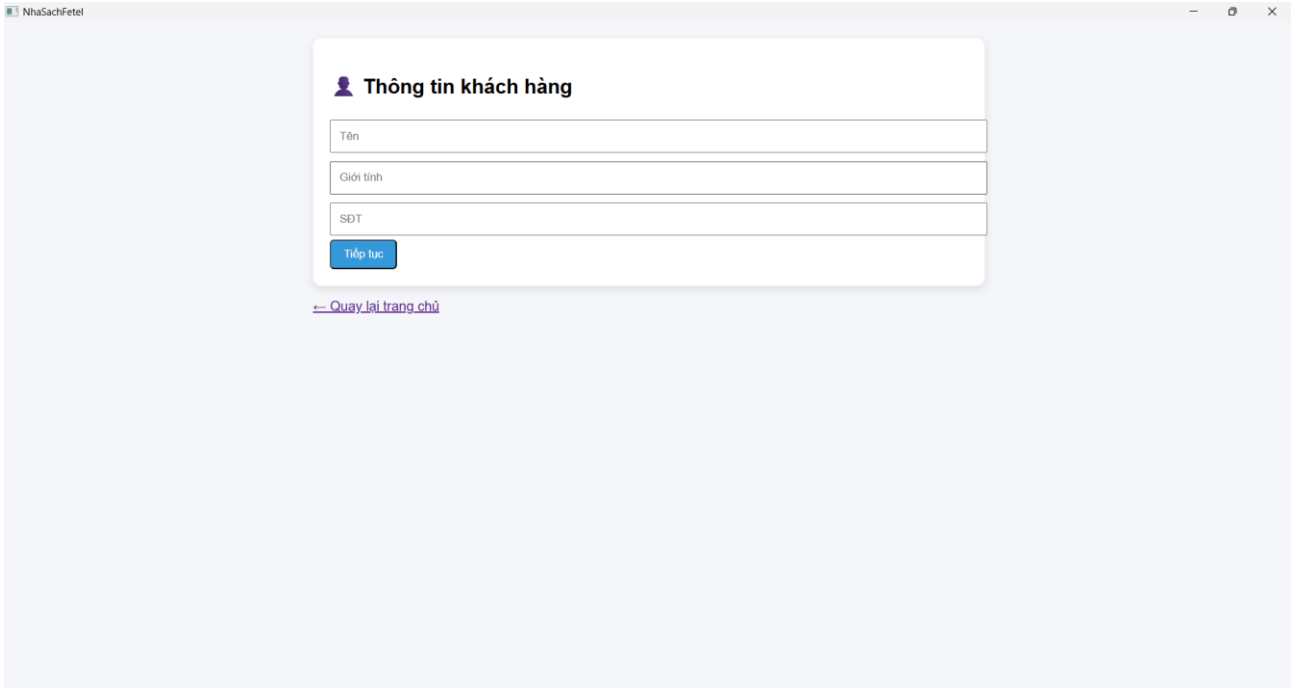


Hình 3. Giao diện quản lý kho sách

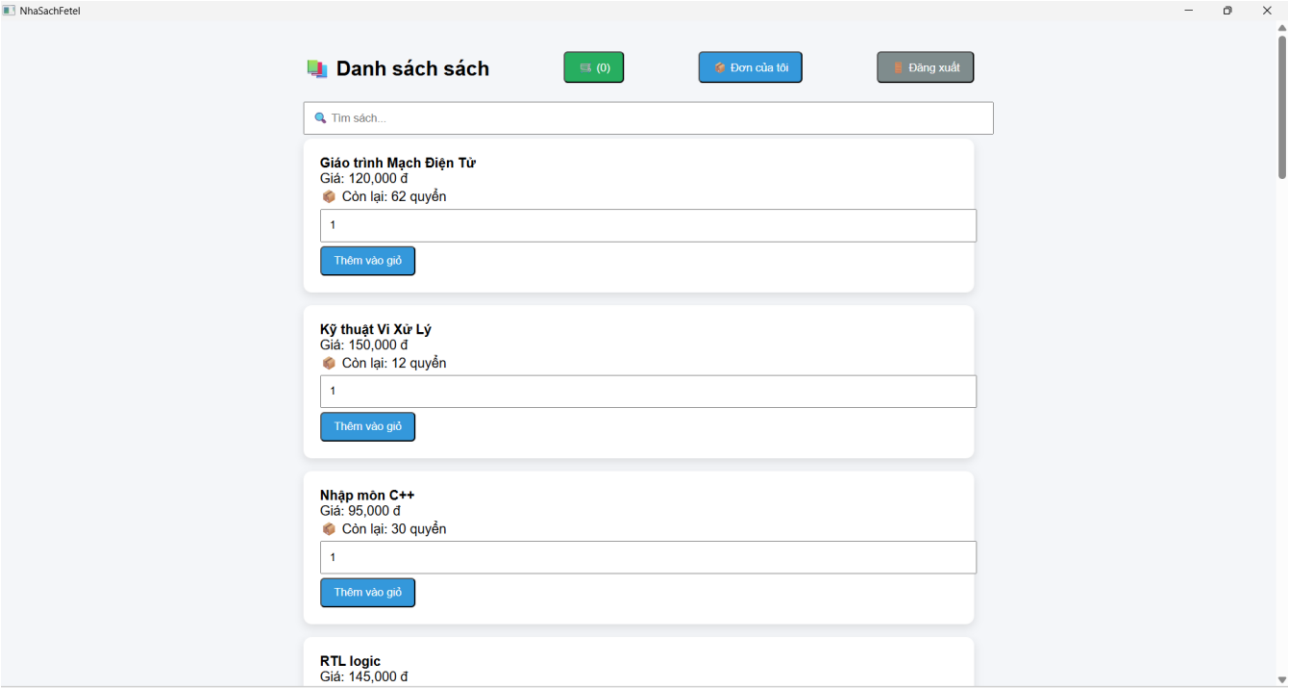


Hình 4. Giao diện quản lý đơn hàng

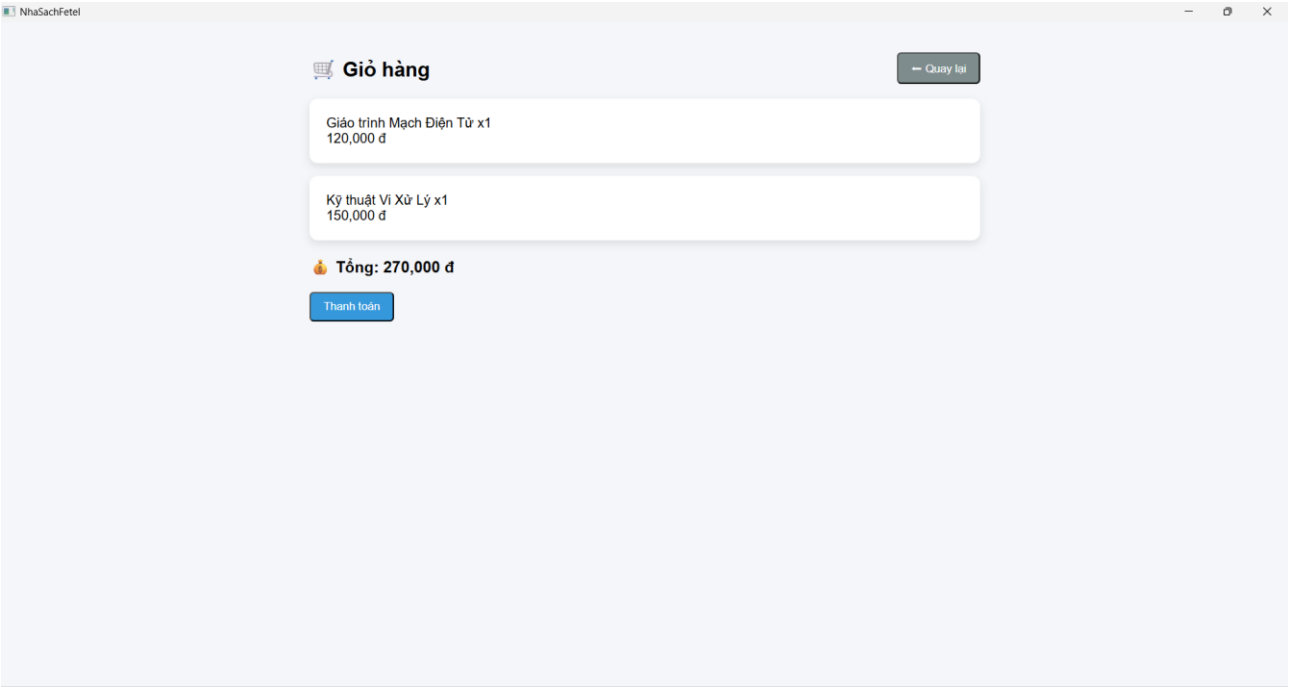
Giao diện tra cứu và mua sắm dành cho Khách hàng.



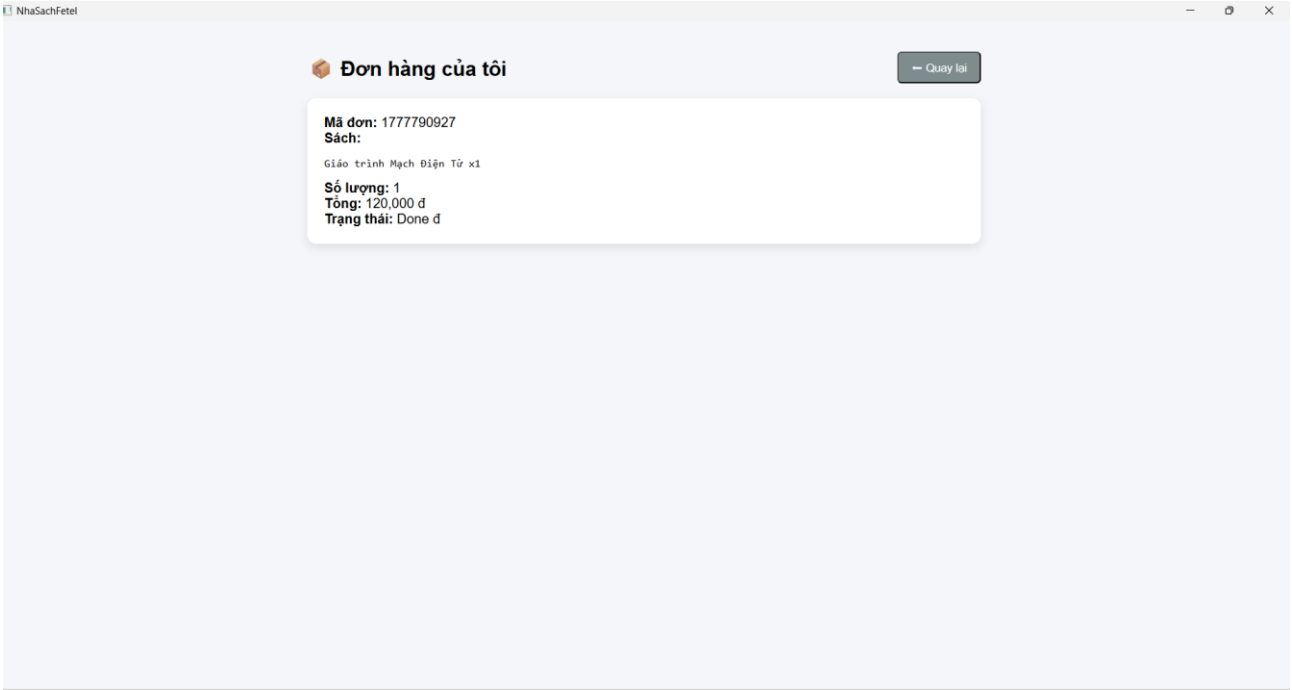
Hình 5. Giao diện đăng ký thông tin khách hàng



Hình 6. Giao diện tra cứu thông tin sách



Hình 7. Giao diện tổng hợp các đơn hàng



Hình 8. Giao diện thông tin chi tiết một đơn hàng

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

1. Kết quả đạt được

Sau quá trình nghiên cứu, thiết kế và cài đặt, đồ án "Xây dựng hệ thống quản lý nhà sách bằng OOP" đã hoàn thành các mục tiêu ban đầu đề ra, cụ thể:

- **Về mặt lý thuyết:** Vận dụng thành công và sâu sắc các nguyên lý của Lập trình hướng đối tượng (OOP) bằng ngôn ngữ C++. Các tính chất Trừu tượng, Đóng gói, Kế thừa và Đa hình được triển khai triệt để, đặc biệt là kỹ thuật sử dụng Interface (IService) để xử lý đa hình trong quản lý luồng dữ liệu tệp tin.
- **Về mặt kỹ thuật:** Xây dựng thành công một kiến trúc ứng dụng Desktop lai (Hybrid) hiện đại. Tích hợp trơn tru logic xử lý mạnh mẽ của C++ (Backend) với giao diện người dùng trực quan bằng HTML/CSS/JS (Frontend) thông qua công nghệ cầu nối WebView2.
- **Về mặt ứng dụng:** Hoàn thiện một phần mềm quản lý nhà sách cơ bản, đáp ứng được các nghiệp vụ thực tế như: quản lý kho sách, định danh khách hàng, xử lý giỏ hàng, xác thực quản trị viên và lưu trữ dữ liệu an toàn dưới định dạng CSV.

Source:https://github.com/MinhQuocNguyenHoang/BookStoreManager

Demo:https://drive.google.com/drive/folders/1hC_1IXQecgSSt132cU__JAUeUztwt

meU?usp=sharing

2. Hạn chế của hệ thống

Mặc dù đã đáp ứng được các yêu cầu cơ bản của môn học, hệ thống hiện tại vẫn còn một số giới hạn nhất định do rào cản về thời gian và khuôn khổ đồ án:

- **Giới hạn về lưu trữ dữ liệu:** Hệ thống sử dụng tập tin phẳng (CSV) để lưu trữ. Khi lượng dữ liệu (sách, đơn hàng) tăng lên hàng chục ngàn bản ghi, tốc độ truy xuất, tìm kiếm và ghi đè toàn bộ tệp sẽ bị giảm sút. Tập tin CSV cũng thiếu vắng các cơ chế bảo toàn dữ liệu phức tạp (ACID) như trong các Hệ quản trị cơ sở dữ liệu quan hệ (RDBMS).
- **Vấn đề bảo mật:** Hiện tại, thông tin đăng nhập của Quản trị viên (Admin) đang được so sánh và xử lý dưới dạng văn bản thuần túy (plain-text). Mật khẩu chưa được mã hóa, dẫn đến rủi ro lộ lọt thông tin nếu mã nguồn hoặc tệp dữ liệu bị truy cập trái phép.
- **Xử lý đồng thời (Concurrency):** Ứng dụng hiện được thiết kế để chạy đơn luồng (single-thread) trên máy cá nhân. Nếu áp dụng vào môi trường thực tế có nhiều máy tính cùng truy cập và chỉnh sửa một tệp CSV, hệ thống sẽ gặp lỗi xung đột đọc/ghi (Read/Write Conflict).

3. Hướng phát triển trong tương lai

Dựa trên những hạn chế đã phân tích, nhóm phát triển đề xuất các hướng nâng cấp hệ thống trong tương lai như sau:

- **Nâng cấp Cơ sở dữ liệu:** Chuyển đổi cơ chế lưu trữ từ tập tin CSV sang các Hệ quản trị cơ sở dữ liệu như SQLite (cho ứng dụng offline nhỏ) hoặc MySQL/SQL Server (nếu mở rộng thành hệ thống Client-Server) để tối ưu hóa tốc độ truy vấn và đảm bảo tính toàn vẹn dữ liệu.
- **Tăng cường bảo mật:** Áp dụng các thuật toán băm (Hashing) như SHA-256 kết hợp với Salt để mã hóa mật khẩu của người dùng.
- **Mở rộng tính năng nghiệp vụ:**
    - Bổ sung thêm phân quyền "Nhân viên bán hàng" (Staff) với các giới hạn truy cập nhất định so với "Quản trị viên" (Admin).
    - Tích hợp tính năng xuất hóa đơn đơn hàng ra tệp PDF để tiện in ấn cho

khách hàng.

    - Xây dựng thuật toán phân trang (Pagination) trên giao diện Web để hiển thị mượt mà khi số lượng đầu sách quá lớn.

- **Tối ưu hóa kiến trúc Frontend:** Tách biệt các đoạn mã JavaScript và CSS hiện đang nhúng trực tiếp trong HTML ra thành các tệp tĩnh (Static files) riêng biệt để tăng tính đóng gói, dễ bảo trì và dễ tái sử dụng.

4. Kết luận chung

Đồ án "Xây dựng hệ thống quản lý nhà sách bằng OOP" không chỉ là một bài tập thực hành kỹ năng lập trình C++ đơn thuần, mà còn là một trải nghiệm toàn diện về quy trình phát triển phần mềm. Việc tự tay thiết kế kiến trúc phân lớp, kết nối Backend với giao diện Web qua WebView, và tự xử lý luồng dữ liệu tập tin đã giúp nhóm củng cố tư duy logic, nắm vững triết lý Lập trình hướng đối tượng. Những kiến thức và kinh nghiệm thu được từ đồ án này chính là bước đệm vững chắc để nhóm có thể tiếp cận và phát triển các hệ thống phần mềm phức tạp, quy mô lớn hơn trong tương lai.

Tài liệu tham khảo

[1] Paul Deitel và Harvey Deitel, *C++ How to Program* (10th Edition), Pearson, 2016. *(Tài liệu tham khảo nền tảng về ngôn ngữ C++ và các nguyên lý lập trình)*.

[2] Microsoft Corporation, *Introduction to Microsoft Edge WebView2*, Microsoft Learn. [Trực tuyến]. Có tại: https://learn.microsoft.com/en-us/microsoft-edge/webview2/ (Truy cập: tháng 5/2026). *(Tài liệu gốc tham khảo cơ chế nhúng giao diện Web vào ứng dụng Desktop bằng C++)*.

[3] Mozilla Foundation, *MDN Web Docs - HTML, CSS, and JavaScript*. [Trực tuyến]. Có tại: https://developer.mozilla.org/ (Truy cập: tháng 5/2026). *(Tài liệu quy chuẩn về thiết kế giao diện và xử lý sự kiện Frontend)*.

[4] Cplusplus.com, *C++ Reference: Input/Output with files*. [Trực tuyến]. Có tại: https://cplusplus.com/doc/tutorial/files/ (Truy cập: tháng 5/2026). *(Tài liệu tham khảo phương thức xử lý thư viện <fstream> cho luồng đọc/ghi tập tin CSV)*.